



BrightLearn
DATA ANALYTICS

EXERCISE 03 · SQL FOUNDATIONS

SQL CASE Statements

A hands-on, pen-and-paper practice exercise. Read each table, write the SQL CASE expression that classifies the rows, and draw the expected output table.

COURSE

BrightLearn Basic SQL

FORMAT

Handwritten · Pen & Paper

QUESTIONS

10 queries across 10 tables

FOCUS

CASE · Classification · Logic



00

Exercise Instructions

This exercise focuses on the **CASE** expression — SQL's way of classifying rows into labelled buckets. Work through each question carefully — accuracy beats speed.

★ LEARNING OBJECTIVES

By the end of this exercise you will be able to:

- Write simple and searched **CASE** expressions inside a **SELECT**.
- Combine **CASE** with comparison operators, **BETWEEN**, **AND**, and **OR**.
- Use **CASE** to classify rows into business categories (price tiers, grades, stock status, etc.).
- Alias the resulting column with **AS** to match the expected output.
- Predict the exact output table of a query before running it.

FOR EACH QUESTION, DO THIS:

1

WRITE THE QUERY

Hand-write the full SQL statement that produces the requested result. Use proper SQL keywords in CAPS.

2

DRAW THE OUTPUT

Draw the expected output table by hand — column headers on top, rows underneath.

3

CHECK COLUMNS

Make sure your output uses the exact column names listed under *Expected Columns*.

SUBMISSION RULES

- **Format:** Handwritten on lined paper, using pen (no pencil, no typing).
- **Layout:** Number every answer to match the question (1 through 10).
- **Queries:** Write each SQL statement neatly. SQL keywords in **UPPERCASE**, table and column names in lowercase.
- **Output tables:** Draw clear column headers, then list each row beneath. Use a ruler if your handwriting needs help staying straight.
- **Submission:** Scan your completed work into a single PDF and email it to rofhiwa@brightlighttutorials.co.za.
- **Originality:** All work must be your own — this exercise is graded on your understanding, not a perfect answer.



01

The products Table

A short product catalogue. Use the price column to bucket each item into a price tier.

► **TABLE** products

product_id	product_name	price
1	Laptop	1200.00
2	Phone	800.00
3	Keyboard	45.00
4	Monitor	300.00
5	Mouse	25.00

01

Classify each product by price into the three tiers below.

- **'Expensive'** if price > 1000
- **'Mid-range'** if price between 100 and 1000
- **'Budget'** if price < 100

EXPECTED COLUMNS:

product_name

price

price_category

02

The orders Table

Customer orders with the amount paid. Label each order by its monetary tier.

► **TABLE** orders

order_id	customer_name	amount
1	Alice	150.00
2	Bob	560.00
3	Charlie	999.99
4	Diana	45.50
5	Ethan	1200.00

02

Label each order by its value.

- **'High Value'** for orders ≥ 1000
- **'Medium Value'** for 500–999.99
- **'Low Value'** for orders < 500

EXPECTED COLUMNS:

customer_name

amount

order_value_category



03

The employees Table

Combine the department and salary columns to label each employee with a seniority tier.

► **TABLE** employees

emp_id	emp_name	department	salary
1	John	IT	85000
2	Sara	HR	60000
3	Mark	IT	75000
4	Lucy	Finance	95000
5	Tom	HR	55000

03

Categorize each employee's position level using both department and salary.

- If in 'IT' and salary > 80000 → 'Senior IT'
- If in 'HR' and salary > 55000 → 'Experienced HR'
- Otherwise → 'Staff'

EXPECTED COLUMNS:

emp_name

department

salary

position_level

04

The students Table

A list of student scores. Convert numeric scores into letter grades.

► **TABLE** students

student_id	student_name	score
1	Anna	92
2	Ben	76
3	Cara	59
4	David	83
5	Ella	68

04

Assign each student a letter grade based on their score.

- ≥ 90 → 'A'
- 80-89 → 'B'
- 70-79 → 'C'
- 60-69 → 'D'
- < 60 → 'F'

EXPECTED COLUMNS:

student_name

score

grade



05

The deliveries Table

Delivery times in minutes. Label each delivery by its performance.

► **TABLE** deliveries

delivery_id	delivery_time_minutes
1	45
2	80
3	30
4	65
5	100

05

Label delivery performance based on the time taken.

- ≤ 30 mins → 'Fast'
- 31-60 mins → 'On Time'
- > 60 mins → 'Late'

EXPECTED COLUMNS:

delivery_id

delivery_time_minutes

performance

06

The tickets Table

Support tickets stored with a numeric priority. Convert the priority into a readable label.

► **TABLE** tickets

ticket_id	issue_type	priority
1	Login issue	1
2	Server down	3
3	Slow system	2
4	Email error	2
5	Password reset	1

06

Convert the numeric priority into a readable label.

- 3 → 'High'
- 2 → 'Medium'
- 1 → 'Low'

EXPECTED COLUMNS:

issue_type

priority

priority_label



07

The attendance Table

Calculate each student's attendance percentage and classify the result.

► **TABLE** attendance

student_id	days_present	total_days
1	45	50
2	30	50
3	48	50
4	25	50
5	50	50

07

Calculate attendance percentage and classify the result.

- $\geq 90\%$ → 'Excellent'
- 75-89% → 'Good'
- $< 75\%$ → 'Needs Improvement'

EXPECTED COLUMNS:

student_id

attendance_percentage

attendance_status

08

The products_inventory Table

Stock levels per product. Label each row with the correct stock status.

► **TABLE** products_inventory

product_id	stock_qty
1	5
2	0
3	25
4	10
5	3

08

Label the stock status of each product.

- 0 → 'Out of Stock'
- 1-5 → 'Low Stock'
- > 5 → 'In Stock'

EXPECTED COLUMNS:

product_id

stock_qty

stock_status



09

The classes Table

Classes with their enrolled student counts. Classify each class by size.

► TABLE classes

class_id	subject	enrolled_students
1	Math	30
2	English	25
3	Science	15
4	Art	5
5	History	20

09

Classify each class by the number of enrolled students.

- $\geq 25 \rightarrow$ 'Large'
- $10-24 \rightarrow$ 'Medium'
- $< 10 \rightarrow$ 'Small'

EXPECTED COLUMNS:

subject

enrolled_students

class_size_category

10

The payments Table

Combine the payment method and amount to decide whether a discount applies.

► TABLE payments

payment_id	amount	payment_method
1	50.00	Card
2	200.00	Cash
3	150.00	Card
4	75.00	PayPal
5	300.00	Cash

10

Apply a discount flag based on the payment method and amount.

- If `payment_method = 'Cash'` and `amount  $\geq$  200` $\rightarrow$ 'Eligible for Discount'
- Otherwise $\rightarrow$ 'Not Eligible'

EXPECTED COLUMNS:

payment_id

payment_method

amount

discount_eligibility



11

Final Checklist

Before you scan and submit, run through this short checklist so nothing is missing.

★ BEFORE YOU SUBMIT — CHECK YOUR WORK

- Did you write all **10** SQL queries?
- Does each output table use the exact column names listed under *Expected Columns*?
- Did you draw a clear output table for every query — even the ones with a single row?
- Did you use **AS** to alias the CASE result with the expected column name?
- Are your queries readable — keywords in CAPS, one clause per line where helpful?
- Did you scan your work as a **single PDF** before emailing?



BrightLearn
DATA ANALYTICS

Now write the **CASE** expressions.

The fastest way to learn SQL is to write it by hand and predict the output before running it. Take your time, sketch your tables clearly, and double-check your column names. You've got this.

★ B R I G H T L E A R N D A T A A N A L Y T I C S ★